# Express, npm & Structure

everything we covered today

Dinesh Rawat

Instructor · Shorter Loop × Snapied

> Notes from today's class. Read them once tonight while it is fresh. Everything here was shown live, so if something does not make sense, it means you missed it, not that it is hard.

## 1 Express: what it actually is

Node can make a web server on its own. It is just painful. You have to read the URL yourself, check the method yourself, set headers yourself.

> **Express**
> A **framework**. It sits on top of Node and does the boring parts for you: reading the URL, matching the method, parsing the body, sending JSON. You write the interesting part.

The whole of Express is basically this idea:

```
app.METHOD(PATH, HANDLER)

app.get('/hello', (req, res) => {
  res.json({ message: 'hi' });
});
```

"When a GET comes to /hello, run this function."

### the two things every handler gets

| Name | What it is |
|---|---|
| `req` | The request. What the client sent you. |
| `res` | The response. What you send back. |

Useful bits of `req`:

- `req.params` — the `:id` part of the URL
- `req.query` — the `?limit=5` part
- `req.body` — the JSON they sent (only if `express.json()` is on)
- `req.method` and `req.url` — GET, POST, and the path

Useful bits of `res`:

- `res.json({...})` — send JSON back
- `res.status(404)` — set the status code
- `res.send()` — send nothing (used with 204)

Alternatives to Express so you know they exist: Fastify (faster), Koa (by the Express team, smaller), NestJS (big, opinionated, structure enforced), Hapi. Express is the most common and has the most answers online. That is why we use it.

## 2 Middleware

**middleware**
Any function that runs **before** your route handler. It gets the request, does one small job, then passes it on.

Think of it as a queue. The request walks through each middleware in order, then reaches your controller.

```
request  ->  express.json()  ->  logger  ->  your controller  ->  response
```

### the shape of every middleware

```
function myMiddleware(req, res, next) {
  // do something
  next();   // done, carry on
}
```

**next()**
Means "I am finished, run the next thing". If you do **not** call it, the request stops right there. That is how a checking middleware blocks a bad request.

### the one we wrote today

**middleware/logger.js**

```
// Our own middleware. Prints every request that comes in.
function logger(req, res, next) {
  console.log(req.method, req.url);
  next();   // done, carry on to the next thing
}

module.exports = logger;
```

Now every request prints itself. Small, but it is a real middleware and you wrote it.

### middleware you did not write

| Middleware | Job |
|---|---|
| `express.json()` | Read the JSON body, put it in `req.body` |
| `cors()` | Let the browser call your API from a page |
| `express.static()` | Serve HTML, CSS, images from a folder |

Order matters. This is the number one Express bug. Middleware runs top to bottom. If `app.use(express.json())` comes after your routes, then `req.body` is `undefined` inside them and you will lose an hour. Put it near the top.

## 3 npm, properly

**npm**
Node Package Manager. Two things at once: a **website** that stores everyone's code (the registry, npmjs.com), and a **command** that downloads it into your project.

### package.json is the whole point

```
npm init -y
```

That makes `package.json`. It is a list of what your project needs. Someone clones your repo, runs `npm install`, and npm reads that file and fetches everything.

This is why we never commit node_modules. It is thousands of files and hundreds of MB. You commit `package.json` instead, which is a few lines, and anyone can rebuild the folder from it. That is what `.gitignore` is for.

### local vs global

| Command | Where it goes |
|---|---|
| `npm install express` | `./node_modules/` — this project only |
| `npm install -g nodemon` | One place on your whole computer |

Rule: if your code `require`s it, install it locally. If it is a tool you type in the terminal, install it globally.

Why does it matter? Two projects might need different versions of Express. Local means each project has its own copy and they never fight. Global means one version for everything, which is fine for a tool, and a disaster for a library.

### dependencies vs devDependencies

```
npm install express          -> dependencies
npm install nodemon --save-dev   -> devDependencies
```

dependencies = your app needs this to run in production.

devDependencies = only needed while developing. Testing tools, nodemon.

On a server, `npm install --production` skips the dev ones. Smaller, faster, fewer things to break.

### semantic versioning (semver)

Every package version is three numbers:

```
   4    .    22    .    2
 MAJOR     MINOR      PATCH
```

| Part | Goes up when |
|---|---|
| MAJOR | They broke something. Your code may stop working. |
| MINOR | They added something. Your code still works. |
| PATCH | They fixed a bug. Nothing else changed. |

Now the symbols in front of the number:

| Symbol | Means |
|---|---|
| `^4.18.0` | Any 4.x. Newest minor and patch. The default. |
| `~4.18.0` | Any 4.18.x. Patches only. |
| `4.18.0` | Exactly this. Nothing else. |

### I tested this in front of you

```
$ npm install express@^4.18.0

package.json says : ^4.22.2
actually installed: 4.22.2
```

Look at what happened. I asked for `^4.18.0`. npm installed 4.22.2, because the caret means "anything below 5.0.0". It did not give me 4.18.0. It gave me the newest 4.x. That is the caret doing its job.

**package-lock.json**

`package.json` says "any 4.x". That is vague, so two people installing on different days can get different versions. The lock file records the exact version you actually got. Commit it. It is why "works on my machine" happens less.

### scoped packages

```
express            <- normal
@types/node        <- scoped
@babel/core        <- scoped
```

The `@something/` part is a scope. It is a namespace, like a folder for package names.

Why they exist: `node` was taken years ago. But `@types/node` is free, because it lives under the `@types` scope. It also tells you who publishes it. Anything under `@types` is from the same team.

Scopes are also how private packages work. At a company you would publish `@shorterloop/utils`, and only people with access can install it.

### public vs private

| Type | Who can install it |
|---|---|
| public | Anyone. Free to publish. |
| private | Only your team. Paid, or your own registry. |

A company keeps internal code private, then installs it in ten projects like any other package. Same npm commands, different permissions.

### alternatives to npm

| Tool | The pitch |
|---|---|
| npm | Comes with Node. No reason not to use it. |
| yarn | Was much faster than npm. npm caught up. |
| pnpm | Saves disk space. One copy shared across projects. |
| bun | Very fast. Newer. Less settled. |

They all read the same `package.json` and pull from the same registry. Learn npm. The others are a small step once you know it.

### commands worth remembering

```
npm init -y                 make package.json
npm install                 install everything in package.json
npm install express         add a package
npm install -g nodemon      install a tool globally
npm uninstall express       remove it
npm view express versions   see every published version
npm outdated                what is out of date
npm root -g                 where global packages live
```

## 4 Use PowerShell, not Command Prompt

I asked you to switch today. Reasons:

- Command Prompt is from the 1980s. It genuinely has not changed much.
- PowerShell understands the commands you see in tutorials. `ls`, `pwd`, `cat`, `touch` mostly work. In cmd they do not exist.
- Every tutorial you find is written for Mac or Linux. PowerShell is closer to those, so you can follow along without translating.
- Better errors, better history, better everything.

> Even better: install Windows Terminal from the Microsoft Store and run PowerShell inside it. Tabs, copy paste that works, and it does not look like 1987. Free.

## 5 The structure we built

```
resume-api/
  app.js          starts the server
  routes/         which URL goes where
  controllers/    the function that answers
  models/         reading and writing data
  middleware/     runs before controllers
```

Yesterday everything was in one file. Today we split it. Here is the honest reason.

You will look for things. A bug in "saving a document" means you open `documentModel.js`. You do not scroll a 400 line file hunting for it.

Next module we swap the file for a database. Then only `models/` changes. Everything else stays untouched. That is the payoff, and it is a big one.

**app.js**

```
const express = require('express');
const routes = require('./routes');

const app = express();

// Middleware: runs before every route
app.use(express.json());

// All our routes live under /api
app.use('/api', routes);

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

That is the whole file. It starts the server. It does not know what a document is.

**routes/index.js**

```
const express = require('express');
const router = express.Router();

router.use('/documents', require('./documentRoutes'));

module.exports = router;
```

One file that lists what the API has. Add a resource, add a line. `app.js` never changes again.

**routes/documentRoutes.js**

```
const express = require('express');
const router = express.Router();

const controller = require('../controllers/documentController');

router.get('/hello', controller.hello);

module.exports = router;
```

**controllers/documentController.js**

```
// GET /api/documents/hello
function hello(req, res) {
  res.json({ message: 'Hello from documents' });
}

module.exports = { hello };
```

That is the hello route, end to end. Three files for one route. It feels like too much today. It stops feeling like that at route number twenty.

```
GET http://localhost:3000/api/documents/hello

{ "message": "Hello from documents" }
```

Follow the path: `app.js` says `/api` goes to `routes/index.js`. That says `/documents` goes to `documentRoutes.js`. That says `/hello` goes to `controller.hello`. So the full URL is `/api` + `/documents` + `/hello`.

## 6 How Shorter Loop does it

I showed you our real backend today. The point was not to impress you. It was this:

The folders are the same. routes, controllers, models. Same names, same jobs. Ours has more files and more layers, because it has been running for years and has real users. But if you opened it looking for how documents get saved, you would know which folder to open. That is not a coincidence. It is the point of doing it this way.

What is different in ours:

- A services folder, where the rules live, so controllers stay thin
- Real authentication instead of a pretend user
- A database instead of `data.json`
- Tests

You will get to all of it. Not this week.

## 7 What models are for

**model**

The only place that touches your data. Reading it, writing it, finding one, deleting one. Nothing else in your app is allowed to touch `data.json` directly.

A model function looks like this:

```
function findById(id) {
  const data = db.read();
  return data.documents.find(doc => doc.id === id);
}
```

Read the file. Find the thing. Give it back. That is all.

Notice what is NOT in there. No `res`. No status code. No 404. The model does not know it is being used by a website. It could be used by a script, a test, anything. It just reads and writes.

Why this matters: next module `db.read()` becomes a database call. This function changes. Nothing else in your app changes, because nothing else knew where the data came from.

## 8 What controllers are for

**controller**

The function that answers the request. It reads `req`, asks a model for data, and sends a response with the right status code.

```
function getOne(req, res) {
  const doc = documentModel.findById(req.params.id);

  if (!doc) {
    return res.status(404).json({ error: 'Document not found' });
  }

  res.status(200).json(doc);
}
```

Three lines of real work. Ask the model. Check the answer. Send it.

### the split, in one sentence each

| Layer | Knows about | Never knows about |
|---|---|---|
| `controller` | req, res, status codes | fs, data.json |
| `model` | the data | req, res, HTTP |

The test for whether you got it right. Is there an `fs` or a `db.read()` in your controller? Then it belongs in the model. Is there a `res.status()` in your model? Then it belongs in the controller. That is the whole rule.

## 9 Postman

**Postman**
An app for calling your API by hand. You pick the method, type the URL, add a body, hit Send. You see the response and the status code.

Why you need it: the browser address bar can only do GET. You cannot POST from it. You cannot send a JSON body from it. Postman can.

### sending a POST

- Set the method to POST
- URL: `http://localhost:3000/api/documents`
- Go to the Body tab
- Choose raw, then pick JSON in the dropdown on the right
- Type: `{ "title": "My CV" }`
- Send

The mistake everyone makes on day one. You choose raw but forget to change Text to JSON in the dropdown. Then `express.json()` ignores your body, `req.body` is empty, and you spend twenty minutes blaming your code. Check the dropdown says JSON.

### what to look at in the response

- The status code, top right. 201 means created. 404 means not found. This is the first thing to read, not the last.
- The body, in the middle.
- Time, next to the status. Useful later.

Make a collection. Save every request you build. Tomorrow you will not retype them. Name them properly: "create document", "list documents". By the end of the module you will have your whole API in one folder you can run in ten seconds.

## 10 For tomorrow

- Have your folders made: `routes`, `controllers`, `models`, `middleware`
- Have the hello route working and hit it from Postman
- Have PowerShell installed and be using it
- Know what your `package.json` says and why

If the hello route works, you have done today's job. It is three files and one line of real code. But you now have a structure that will hold thirty routes without becoming a mess. That is what we built today. Tomorrow we fill it in.